

Logminer: A Lightweight Multi-Agent Framework for Family-Aware Multi-Model Anomaly Detection in Heterogeneous Logs

Author Name(s) Omitted for Review

Department / Laboratory

University / Organization

Email: author@example.com

Abstract—Security monitoring environments generate heterogeneous logs whose formats, semantics and statistical distributions differ across operating systems, network flows, SIEM exports and distributed systems. Applying a single monolithic detector to all sources may increase confusion between log families and reduce the interpretability of alerts. This paper presents Logminer, a lightweight multi-agent software architecture for anomaly detection in heterogeneous system and network logs. The prototype decomposes the workflow into specialized cooperating agents for collection, parsing, normalization, family-aware routing, anomaly detection, correlation, dashboard visualization and audit. The main contribution is an agentic family-aware multi-model routing strategy: events are normalized into a common schema, assigned to a log family, processed by a specialized lightweight model, then unified as anomaly candidates before being correlated into analyst-readable incidents. Initial experiments cover Windows logs, Wazuh exports, Linux/auth logs, HDFS, BGL, CICIDS2017 and UNSW/CIC-DDoS data. The results support the feasibility of a local lightweight implementation and highlight the need to evaluate family-aware specialization against monolithic baselines. The prototype is positioned as an analytical complement to mature log management and HIDS/SIEM tools, not as a replacement for production SOC platforms.

Index Terms—anomaly detection, log analysis, multi-agent architecture, cybersecurity, family-aware routing, lightweight machine learning, SIEM complement, log correlation

I. INTRODUCTION

System and network logs are central to security monitoring, incident investigation and operational audit. However, modern infrastructures generate logs from heterogeneous sources: Windows event logs, Linux authentication logs, SIEM/HIDS exports, network-flow datasets, cloud logs and distributed-system traces. These sources differ not only in syntax, but also in event semantics, feature availability, temporal density and class imbalance. This heterogeneity complicates the design of a single anomaly detection pipeline.

Classical intrusion detection and log management research has long emphasized audit trails, deviation from normal behavior and the difficulty of reducing false positives [1], [2], [4]. More recent machine learning approaches provide promising results, but remain sensitive to dataset shift, closed-world assumptions, class imbalance and operational interpretability [3], [5], [6]. In practice, operational tools such as fail2ban, OSSEC, Wazuh or ELK-like stacks provide valuable collection, rule-based detection, indexing and visualization, but they

do not by themselves solve the problem of controlled model specialization across heterogeneous log families.

This paper introduces Logminer, a lightweight multi-agent prototype designed to evaluate a family-aware multi-model strategy for heterogeneous log anomaly detection. The term multi-agent is used here in a software architectural sense: Logminer is composed of specialized cooperating agents with distinct functional roles, message exchange and workflow coordination. The prototype does not claim to implement fully autonomous cognitive agents. Instead, it establishes a modular and extensible basis for more autonomous decision support.

The contribution of this paper is threefold:

- A lightweight multi-agent architecture for log analysis, including collector, parser, normalizer, detector, correlator, dashboard and audit roles.
- A family-aware multi-model routing strategy in which normalized logs are assigned to a log family and processed by a specialized model rather than by a single monolithic detector.
- An initial multi-source evaluation covering system logs, SIEM exports and network datasets, with discussion of false positives, quasi-real-time workflow latency, resource consumption and operational limits.

The paper deliberately uses cautious claims. Non-supervised outputs are treated as anomaly candidates, not confirmed intrusions. The validated distribution is mainly logical and modular, while large-scale multi-machine distribution remains future work.

II. RELATED WORK

A. Anomaly Detection and Intrusion Detection

Denning's intrusion detection model introduced the idea of detecting deviations from expected user and system behavior through audit records [1]. Axelsson later structured IDS concepts and taxonomies, while Chandola et al. provided a broader survey of anomaly detection methods across point, contextual and collective anomalies [2], [4]. These foundations are directly relevant to log anomaly detection, where isolated rare events and correlated sequences may indicate abnormal behavior.

Machine learning has been widely investigated for cyber intrusion detection. Buczak and Guven review data mining and

machine learning methods for cybersecurity, while Khraisat et al. discuss IDS datasets, challenges and evaluation issues [5], [6]. Sommer and Paxson warn against overinterpreting machine learning results in intrusion detection, especially when training and testing assumptions do not reflect operational environments [3]. This warning motivates the cautious positioning adopted in this work.

B. Log Parsing and Log-Based Detection

Before machine learning can be applied, raw logs must be parsed and transformed into usable representations. Template-based approaches such as Drain organize log messages into event templates and enable downstream sequence or statistical modeling [7]. DeepLog and related approaches model log sequences with deep learning to detect deviations from normal execution patterns [8]. Logminer currently uses lightweight parsing and normalization, while template-based parsing is retained as a natural extension for Linux, HDFS and BGL-style logs.

C. Agent-Based Security Monitoring

Agent-based security monitoring has historical roots in distributed intrusion detection. AAFID proposed an architecture using autonomous agents to collect and analyze host-level events before aggregation [9]. Logminer follows this lineage in a more modest way: agents are software modules and services with clear responsibilities and exchange contracts. The goal is not full cognitive autonomy, but controlled specialization, modular replacement and future deployability.

III. PROPOSED ARCHITECTURE

A. Architectural Scope

The proposed architecture follows a log processing chain from raw sources to analyst-readable incidents. Fig. 1 summarizes the logical workflow. Each agent has a restricted responsibility: the collector retrieves or receives raw logs; the parser detects formats and extracts fields; the normalizer maps extracted fields to a common schema; the router assigns events to a log family; the detector applies a family-specialized model; the correlator groups anomaly candidates into incidents; and the dashboard exposes results for human validation and audit.

The distribution validated in this work is primarily logical and modular. The service-level implementation is partially supported through FastAPI endpoints, a local JSONL bus and an optional Redis-based evolution. A large-scale multi-machine deployment is not claimed as a completed result.

B. Agent Roles and Interaction

Table I summarizes the main agents. This decomposition is important because it prevents the detector from becoming an opaque monolithic block. It also allows future replacement of one agent without redesigning the entire pipeline.

Communication is implemented through file-based artifacts, JSONL messages, FastAPI endpoints and optional Redis evolution. Coordination is handled by an orchestrated workflow,

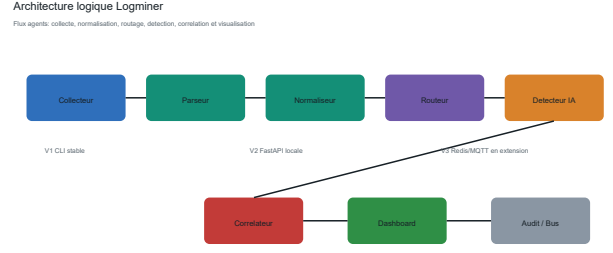


Fig. 1. Logical architecture of Logminer. Heterogeneous logs are collected, parsed, normalized, routed to specialized models, transformed into anomaly candidates and correlated into analyst-readable incidents.

TABLE I
SPECIALIZED AGENTS IN THE LOGMINER ARCHITECTURE

Agent	Role	Output
Collector	Retrieves local or exported logs from Windows, Linux, SIEM or dataset folders	Raw files and manifests
Parser	Detects format and extracts timestamp, host, event, severity, source and message fields	Structured events
Normalizer	Maps heterogeneous fields to a common event schema	Normalized records
Router	Identifies the family of each log batch or event stream	Model selection decision
Detector	Applies a lightweight supervised or non-supervised model specialized by family	Anomaly candidates
Correlator	Groups anomalies by time, source, host, severity or context	Interpretable incidents
Dashboard/Audit	Displays events, anomalies, incidents and analyst decisions	Visual analysis and traceability

including discovered-source execution and route-based processing.

C. Normalized Event Contract

The common schema contains fields such as timestamp, severity, event identifier, source, host, user, source and destination IP, ports, category, subcategory and message. This common representation does not erase family-specific information; it provides the minimal interface required for model routing, scoring, correlation and visualization. The design follows log management principles in which normalization and retention of raw context are both necessary for auditability [10], [11].

IV. FAMILY-AWARE MULTI-MODEL ROUTING

A. Motivation

The central hypothesis of this work is that log families exhibit sufficiently different structures and distributions to justify specialized detection models. Windows event logs, Linux/auth logs, Wazuh alerts, HDFS traces, BGL HPC logs and network-flow datasets do not expose the same fields, semantics or anomaly patterns. A single global model may

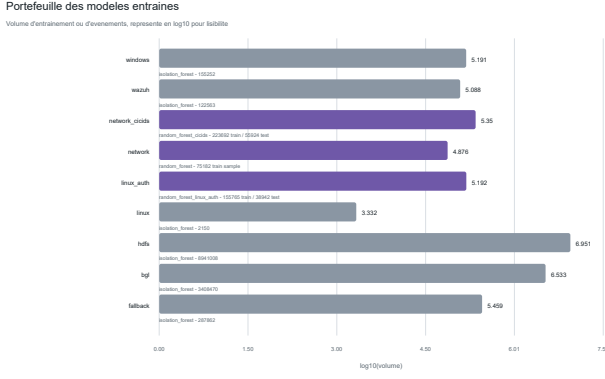


Fig. 2. Portfolio of model artifacts used by Logminer across Windows, Wazuh, Linux/auth, network, HDFS, BGL and fallback families.

TABLE II
ABLATION PROTOCOL REQUIRED TO VALIDATE FAMILY-AWARE ROUTING

Configuration	Purpose
Global model	Main baseline: one detector applied to all normalized sources
Family-aware models	Measures the effect of specialized detectors by log family
Without correlation	Isolates the impact of the routing and detector agents
With correlation	Measures the complete pipeline from anomaly candidate to incident
Without common normalization	Estimates the cost of heterogeneous feature handling
With common normalization	Tests the complete proposed pipeline

therefore learn dominant distributions from one family and behave poorly on another.

Logminer addresses this issue through agentic family-aware routing. After normalization, each batch or stream is associated with a log family. The router selects a model compatible with that family. The detector then produces anomaly candidates in a unified output format, allowing downstream correlation and dashboard analysis.

B. Scientific Claim

The novelty is not the isolated use of parsing, machine learning or a dashboard. The novelty is a strategy of controlled specialization: heterogeneous logs are normalized, routed according to family, scored by specialized lightweight models and correlated into incidents. This strategy aims to improve robustness and analyst usability in the presence of heterogeneous formats and distributions; this hypothesis must be evaluated experimentally against monolithic baselines.

C. Ablation Protocol

To make the claim testable, the final experimental protocol must include an ablation study. Table II defines the required comparisons. The current draft reports initial multi-source results; the ablation metrics must be completed before submission if the article claims empirical superiority of family-aware routing.

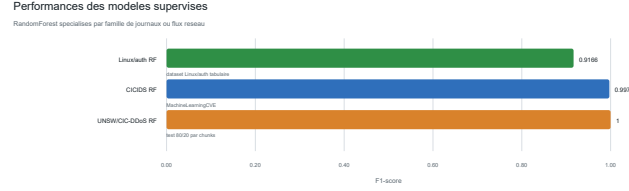


Fig. 3. F1-score of supervised Random Forest models on Linux/auth, CICIDS2017 and UNSW/CIC-DDoS families.

V. IMPLEMENTATION

The prototype is implemented in Python with a modular source tree. The stable version uses command-line execution, CSV artifacts and local folders. The service-oriented version exposes core workflow actions through FastAPI, including source discovery, detection and correlation. A local JSONL bus supports message-oriented traces, while Redis is documented as an optional extension.

The detector portfolio includes supervised Random Forest models for labeled families such as Linux/auth, CICIDS2017 and UNSW/CIC-DDoS, as well as non-supervised Isolation Forest or ensemble-style detectors for sources where labels are incomplete or absent. Non-supervised outputs are explicitly called anomaly candidates.

A web dashboard supports exploration of anomalies, incidents, counters, filters, timeline-style views and analyst decisions. This dashboard is an exploitation layer: it does not prove the anomaly by itself, but helps analysts validate, reject or reinterpret candidate incidents.

VI. DATASETS AND EXPERIMENTAL SETTING

Table III lists the families used in the initial evaluation. The selection mixes local logs, SIEM-style exports, public system logs and public network datasets.

The evaluation reports F1-score where labels are available, false positive rates where confusion matrices are available, workflow latency and CPU/RAM snapshots. For unlabeled SIEM exports and operational logs, results are interpreted as anomaly candidates rather than confirmed intrusions.

VII. INITIAL RESULTS

A. Detection Results

Table IV summarizes the principal results available in the current prototype. The supervised Random Forest models obtain strong F1 scores on CICIDS2017 and UNSW/CIC-DDoS, and a lower but still usable score on Linux/auth. HDFS remains more difficult with the current lightweight features, whereas BGL obtains high validation scores.

These results should not be interpreted as universal superiority. They show that the prototype can integrate several families and that family-specific models are operationally feasible. The ablation protocol in Table II is required to isolate the effect of the routing strategy from dataset-specific model performance.

TABLE III
DATASETS AND SCENARIOS USED IN THE INITIAL EVALUATION

Category	Source	Nature	Objective
Local real	Windows Event/Application/System/Security	Windows system logs	Test EVT/XML collection, parsing and local candidate detection
Real/SIEM	Wazuh exports	SIEM/HIDS events and alerts	Test non-supervised detection on operational security signals
Real tabular	Linux/auth	Labeled authentication logs	Supervised evaluation and false positive analysis
Public system	HDFS	Distributed-system logs with labels	Validate detectors on sequential system logs
Public system	BGL	BlueGene/L logs with labels	Validate detectors on HPC logs
Public network	CICIDS2017	Labeled network flows	Supervised detection of known attack classes
Public network	UNSW/CIC-DDoS	Network/DDoS flows	Chunk-based 80/20 evaluation
Robustness	Apache, CEF/LEEF, CloudTrail, Linux auth, incomplete log	Multi-format and corrupted input	Test parser tolerance and preservation of unknown records

TABLE IV
PRINCIPAL DETECTION RESULTS AVAILABLE IN THE PROTOTYPE

Family	Dataset	Model	Main result
Windows	Security/Application/System	Isolation Forest	Anomaly candidates and correlated incidents
Wazuh	January/October/December exports	Isolation Forest	122,563 events and 3,676 anomaly candidates
Linux/auth	Authentication logs	Random Forest	F1 = 0.916602
CICIDS	CICIDS2017 MachineLearningCVE	Random Forest	F1 = 0.997163
UNSW/CIC-DDoS	UNSW-NB15/CIC-DDoS network data	Random Forest	F1 = 0.999965
HDFS	Distributed-system logs	Ensemble / Isolation Forest	Best selection around F1 = 0.599333
BGL	BlueGene/L HPC logs	Ensemble / Autoencoder / Isolation Forest	Best selection around F1 = 0.994333

TABLE V
REPRESENTATIVE FALSE POSITIVE RESULTS

Dataset	Model	Test lines	FPR	FP/1000
Linux/auth	Random Forest	38,942	0.047250	29.120
CICIDS2017	Random Forest	55,924	0.001933	1.037
UNSW/CIC-DDoS	Random Forest	1,474,193	0.001709	0.001
BGL	Ensemble selected	6,000	0.005667	2.833
HDFS	Ensemble selected	6,000	0.400667	200.333
Simulated Windows	Isolation Forest	6,000	0.000000	0.000

B. False Positives

False positives matter because they directly affect analyst workload. Table V reports representative false positive rates. HDFS shows a high false positive rate with the current lightweight representation, which supports the need for better templates, sequential features or family-specific feature engineering.

C. Robustness and Workflow Performance

The parser and normalizer were tested on Apache, CEF/LEEF, CloudTrail, Linux/auth and incomplete logs. The objective is not to claim full universal parsing, but to verify that unexpected or corrupted lines are preserved as unknown records without interrupting the workflow.

The FastAPI quasi-real-time workflow was executed for five cycles with 8,537 lines per cycle. The mean workflow latency was 10.6473 seconds, with a minimum of 3.0865 seconds and a maximum of 19.8163 seconds. A 30-cycle resource campaign reported a mean workflow duration of

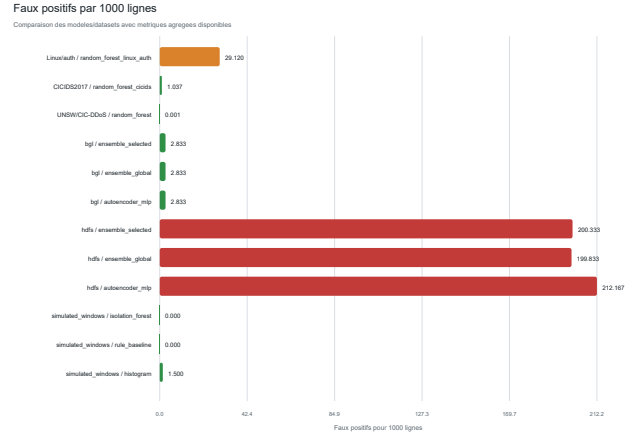


Fig. 4. False positive rates observed by log family and model.

9.3300 seconds and maximum duration of 21.6007 seconds. The API/orchestrator process averaged 187.82 MB RAM and reached 221.20 MB, while the Logminer process averaged 495.41 MB and reached 655.39 MB. These measurements support feasibility on a local machine, but do not establish industrial real-time performance.

VIII. DISCUSSION

A. Comparison with Standard Tools

A common reviewer objection is that ELK, OSSEC, Wazuh or fail2ban already process logs. This paper does not dispute

Robustesse du parsing multi-format
Contrôle sur Apache, CEF/LEEF, CloudTrail, Linux auth et log incomplet

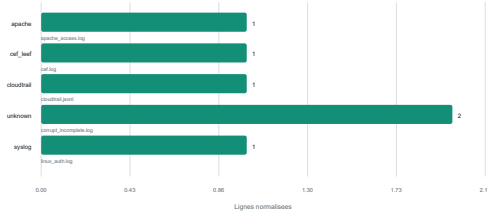


Fig. 5. Robustness check on multi-format inputs. Unknown or incomplete entries are preserved instead of breaking the pipeline.

TABLE VI
QUALITATIVE POSITIONING AGAINST STANDARD TOOLS

Criterion	Logminer	fail2ban	OSSEC/Wazuh
Detection	Statistical/AI anomalies and contextual correlation	Rules on authentication failures and IP banning	Rules, decoding and HIDS/SIEM alerts
Sources	Windows, Linux, Wazuh, HDFS, BGL, network, cloud	Mainly exposed services and auth logs	Broad source coverage through agents and decoders
Adaptation	Multi-model routing by family	Manual filters and thresholds	Rules, lists and SOC configuration
Positioning	Lightweight analytical complement	Immediate operational protection	Mature production HIDS/SIEM base

their operational value. ELK-like stacks mainly provide ingestion, indexing, search, storage and visualization. Wazuh and OSSEC provide mature HIDS/SIEM functions based on agents, decoders and rules. fail2ban provides direct operational reaction to repeated authentication failures. Logminer is different in scope: it focuses on analytical specialization, dynamic model selection by log family, unification of supervised and non-supervised outputs as anomaly candidates, and correlation into analyst-readable incidents.

B. Scope and Limitations

Table VII states the defensible scope of the prototype. This scope is necessary to avoid overclaiming.

C. Threats to Validity

The main internal validity threat is that current results combine multiple datasets, feature representations and model families. Without the ablation study, one cannot attribute performance gains solely to family-aware routing. External validity is limited by the selected datasets and local execution environment. Operational validity is limited because non-supervised anomalies require analyst confirmation, and because a real SOC deployment would require stronger ingestion guarantees, access control, retention policies and response workflows.

IX. CONCLUSION AND FUTURE WORK

This paper presented Logminer, a lightweight multi-agent prototype for heterogeneous log anomaly detection. Its main contribution is a family-aware multi-model routing strategy

TABLE VII
SCOPE AND LIMITATIONS OF THE PROTOTYPE

Point	Defensible position
Multi-agent	Specialized cooperating software agents, not fully cognitive autonomous agents
Distribution	Logical and local service distribution; multi-machine deployment remains future work
Adaptive AI	Adaptation by family-aware multi-model routing, not continuous self-learning
Real time	Local quasi-real-time workflow, not strict industrial SOC real time
Tool comparison	Analytical complement to Wazuh/fail2ban/OSSEC, not replacement
Unsupervised results	Anomaly candidates, not confirmed intrusions

orchestrated by specialized software agents and completed by common normalization, anomaly candidate unification and interpretable correlation. Initial results across Windows, Wazuh, Linux/auth, HDFS, BGL, CICIDS2017 and UNSW/CIC-DDoS support the feasibility of the approach on a local infrastructure.

The next step is to complete the ablation study comparing a global monolithic detector with the proposed family-aware multi-model strategy, with and without correlation and normalization. Further work includes stronger template-based parsing, multi-machine deployment, active learning from analyst decisions, richer incident explanations and integration with mature SIEM/HIDS platforms.

REFERENCES

- [1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.
- [2] S. Axelsson, "Intrusion detection systems: A survey and taxonomy," Technical Report, 2000.
- [3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [5] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys and Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [6] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 20, 2019.
- [7] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *IEEE International Conference on Web Services*, 2017, pp. 33–40.
- [8] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *ACM CCS*, 2017, pp. 1285–1298.
- [9] J. S. Balasubramanian, J. O. Garcia-Fernandez, D. Isacoff, E. Spafford, and D. Zamboni, "An architecture for intrusion detection using autonomous agents," in *Annual Computer Security Applications Conference*, 1998, pp. 13–24.
- [10] National Institute of Standards and Technology, "Guide to computer security log management," NIST Special Publication 800-92, 2006.
- [11] R. Gerhards, "The Syslog Protocol," RFC 5424, 2009.